Review 1: Introduction and Literature Review

Project 14: Natural Language Mount and Partition Agent
Operating Systems Lab, Department of Computer Science and Engineering
August 2026

About this report

This is the first review of our project for the Operating Systems lab. The aim of the project is a terminal program that accepts requests in plain English, such as “mount this ubuntu.iso so I can look at the files inside”, and turns them into the correct mount and partition operations for the system it is running on. This review explains why we chose the problem, what we plan to build and what is outside its scope. It then looks at the background we read before starting: how large language models generate commands, the Model Context Protocol, and the AI command line tools that already exist.

The review template is written for a general natural language shell. Our project is one agent in the OS project series and only deals with disks, partitions and disk images, so the examples in this report are about mounting rather than general shell use.

Chapter 1: Introduction

1.1 Project Overview

1.1.1 The evolution of command line interfaces

The command line is the oldest way of talking to an operating system that is still in daily use. Early Unix systems in the 1970s gave users a shell that read one command at a time, and the Bourne shell later added variables, loops and scripts. Bash, released in 1989, became the default on most Linux distributions and is still what most students first meet. On Windows the story went from COMMAND.COM to cmd.exe and then to PowerShell in 2006, which passes objects between commands instead of plain text.

Shells have become more powerful over the years, but the basic interaction has not changed. The user has to know the exact name of the program, its options and the order of its arguments. Newer shells such as zsh and fish added tab completion, suggestions from history and syntax highlighting. These help people who already roughly know the command they want, but they do not help someone who only knows what they want done.

1.1.2 The shift toward natural language in system administration

Since about 2021, large language models have become good enough at writing code that they can produce shell commands from an English description. Tools like GitHub Copilot in the CLI and Warp’s AI features now let a user describe a task and get a command back. Cloud providers have added similar assistants to their consoles. Research datasets such as NL2Bash and the NLC2CMD competition showed that translating English to Bash is possible, though far from perfect.

Disk management is a good place to apply this idea. Mounting an ISO on Linux takes several steps: find a free loop device with losetup, check the filesystem type with blkid, create a mount point and

finally call mount with the right options. On Windows the same job uses Mount-DiskImage and then Get-Volume to find the drive letter. A mistake with the wrong device name can damage data. A user who just says what they want, and lets the agent pick and check the commands, avoids most of this.

1.2 Problem Statement

1.2.1 Syntax barriers and the learning curve of traditional shells

For students and new administrators the main problem is not understanding what mounting is, but remembering how to do it. The commands below all relate to mounting one disk image on Linux, and each has its own options:

```
lsblk -f
sudo losetup --find --show --partscan --read-only ubuntu.iso
sudo blkid /dev/loop0
sudo mkdir -p /mnt/ubuntu
sudo mount -o ro,nosuid,nodev /dev/loop0 /mnt/ubuntu
sudo umount /mnt/ubuntu && sudo losetup -d /dev/loop0
```

PowerShell uses a completely different set of commands for the same job, such as Mount-DiskImage, Get-Disk, Get-Partition and Add-PartitionAccessPath, and they need to be piped together correctly. Someone who has learned one system has to start again on the other. Manual pages explain every option but do not say which ones matter for a particular task. As a result most people search the web and copy commands they do not fully understand.

1.2.2 Risk of running the wrong command

Disk commands are some of the most dangerous commands in an operating system because they work below the filesystem. Typing /dev/sda instead of /dev/sdb, mounting over an existing directory such as /etc, or forgetting the read-only option on an untrusted image can all cause damage or a security problem. Commands like mkfs and dd do not ask for confirmation, and a single wrong character can wipe a disk.

A plain natural language front end does not remove this risk. If a language model writes a raw shell command and the program runs it directly, any mistake by the model is executed with root privileges. The problem we want to solve is therefore twofold: let users ask in plain English, and make sure that whatever the model produces cannot do more than the user intended.

1.3 Objectives

The objectives of the project are:

- To build a lightweight and secure translation engine that turns English requests about disks, partitions and disk images into the correct operations on Linux and Windows.
- To limit the agent to a small set of typed tools (list devices, list attached images, inspect a filesystem, mount, unmount and list files) instead of letting it write free-form shell commands.
- To describe each tool with a name, a short description and a JSON schema for its arguments, which is the same tool format the Model Context Protocol (MCP) uses, so that the tools can later be offered to other agents as an MCP server.

- To keep the context of a session, so that follow-up requests like “unmount it” or “show me the files” refer to the image mounted earlier.
- To enforce safety in code rather than in the prompt: read-only mounts by default, mount points restricted to safe folders, protection of the system disk, and a confirmation before any change.
- To show the user every command and system call that was run, so the program also helps them learn the underlying operations.
- To work without internet through an offline rule-based parser, with Claude or a local model through Ollama as optional backends.

1.4 Scope and Limitations

1.4.1 Supported operating systems and terminals

Item	In scope	Not in scope
Operating systems	Ubuntu and Debian based Linux, Windows 10 and 11	macOS, BSD, mobile systems
Terminals	Bash and other Linux terminals, PowerShell, Command Prompt, Windows Terminal	Graphical file managers
Image formats	ISO, IMG and raw images on Linux; ISO, VHD and VHDX on Windows	qcow2, VMDK and other virtual machine formats
Filesystems	Whatever the OS can mount, for example ext4, vfat, NTFS, exFAT, iso9660	Repairing or converting filesystems
Operations	Listing, inspecting, mounting, unmounting and listing files	Creating, resizing, formatting or deleting partitions

Table 1.1: Scope of the project

The program will be written in Python 3 and should need no extra packages in offline mode, so it can run on lab machines as they are. Linux support is the main target because the course is based on Linux system calls. Windows support is included because most students use Windows laptops, and it lets us compare how the two systems handle the same task.

1.4.2 Privileges given to the AI model

The language model never gets a shell and never sees a password. It can only ask for one of the six tools by name, with arguments that are checked against the tool’s schema before anything runs. The tools themselves decide which commands to run. This means the model cannot format a disk, delete files or write raw data, because no tool exists for those actions.

Mounting needs root on Linux, so the program has to be started with sudo there. On Windows an ordinary user can mount ISO files, but VHD files and real partitions need an administrator. The program will check its privileges when it starts and tell the user what it can and cannot do. Even when running as root, every mount and unmount asks the user to confirm, and mounts are read-only unless the user asks for write access.

The main limitations we expect are that an offline parser will only understand phrasings it was written for, that results from an online model depend on the network and on the model itself, and that real partitions can only be tested on machines where we are allowed to attach drives.

Chapter 2: Literature Review and Theoretical Background

2.1 Large Language Models in Code Generation

2.1.1 Capabilities in structured text and code

A large language model is a neural network, usually a transformer, trained to predict the next token of text on a very large corpus. Because public code is part of that corpus, these models learn the syntax of programming languages and the common patterns of shell commands. Chen et al. (2021) showed with Codex that a model trained on GitHub code could solve a large share of simple programming problems from their descriptions. Later models improved on this considerably.

Work focused on the command line came earlier. Lin et al. (2018) built NL2Bash, a dataset of about 9,000 English descriptions paired with Bash commands, and the NLC2CMD competition (Agarwal et al., 2021) used it to compare translation systems. Accuracy on these benchmarks was limited, mainly because the same task can be written many ways and small differences in options change what a command does.

Modern models can also produce structured output such as JSON when asked to, and most providers now support tool calling directly. Instead of writing a command as text, the model returns the name of a function and its arguments, and the application decides what to do with them. Schick et al. (2023) and Yao et al. (2023) showed that models can learn when to call a tool and can use the result of one call to decide on the next. This is the approach we plan to use, since it keeps the decision about which commands run inside our own code.

2.1.2 Hallucination in system level execution

Language models sometimes produce output that sounds correct but is wrong. Ji et al. (2023) call this hallucination. For command generation it can mean an option that does not exist, a device name that was never mentioned, or a path that looks plausible but is not on the system. In normal text a hallucination is just an error, but in a terminal running as root it can destroy data.

Other problems we expect are that the model may ignore part of a request, assume Linux when it is running on Windows, or produce output that is not valid JSON. The literature suggests a few common defences: limit the actions available to the model, validate every argument before using it, ask the user before changes, and return errors to the model so it can correct itself. We plan to use all four, so that a wrong answer from the model leads to an error message rather than a wrong command.

2.2 Introduction to the Model Context Protocol (MCP)

2.2.1 Architecture: hosts, clients, servers and tools

The Model Context Protocol is an open standard that Anthropic released in November 2024 for connecting AI applications to outside data and tools. It uses JSON-RPC 2.0 messages, sent either over standard input and output for a local program or over HTTP for a remote one. Its main parts are:

- Host: the application the user interacts with, such as a chat app, an IDE or a terminal program.
- Client: a connector inside the host that keeps one session open with one server.

- Server: a program that offers capabilities to the model. A server can provide tools (actions the model can call), resources (data it can read) and prompts (templates).
- Tool: a named action with a description and a JSON schema for its input. The client lists the available tools, the model chooses one, and the server runs it and returns the result.

A simplified tool definition for our mount operation would look like this:

```
{
  "name": "mount",
  "description": "Mount an image file or partition, read-only by default",
  "inputSchema": {
    "type": "object",
    "properties": { "source": {"type": "string"}, "read_only": {"type":
"boolean"} },
    "required": ["source"]
  }
}
```

2.2.2 Why MCP suits local system access

Before MCP, every AI application connected to tools in its own way, often through custom REST APIs or plugins made for one model provider. That means writing the same integration again for each application and each provider. With MCP the tool is written once as a server, and any MCP client can use it.

MCP also suits our safety goals. The model only sees a list of tools and their schemas and can only ask for those tools, so the server stays in control of what is actually executed. A local server runs as a separate process with its own permissions, which fits the idea of keeping privileged mount operations away from the model. Tools can be discovered at run time rather than being fixed in the prompt. And because the OS project series has many agents, a common protocol would let a general assistant use our mount tools alongside the others.

For this project we will first define the tools inside our own program using the MCP tool format, because that is enough to test the agent and does not need a separate server. Publishing them as a real MCP server is planned for a later stage.

2.3 Existing Solutions

We looked at existing tools that turn natural language into terminal commands, and at the tools people normally use to manage disks. Table 2.1 compares them on the points that matter for our project.

Tool	How it works	Runs commands	Works offline	Disk safety
GitHub Copilot in the CLI	Suggests and explains shell commands from a description using GitHub's models	After the user approves	No	General purpose; no checks specific to disks
Warp AI	Terminal with built-in AI that suggests commands and can run multi-step tasks	After the user approves	No	General purpose; relies on the user reviewing commands

Tool	How it works	Runs commands	Works offline	Disk safety
Shell-GPT (sgpt)	Open-source tool that sends the request to a chosen model and prints a command	Optional, after a prompt	With a local model	None beyond the prompt
Open Interpreter	Open-source agent that writes and runs code and shell commands in a loop	Yes, can run automatically	With a local model	Asks before running by default, but the model can run any code
GNOME Disks / Disk Management	Graphical tools for disks and partitions	Yes, through menus	Yes	Confirmation dialogs; no natural language

Table 2.1: Comparison of existing tools

GitHub Copilot in the CLI and Warp are polished and work well for general shell tasks, but both need an internet connection and a sign-in, and both generate free-form commands. The user is expected to check each command before running it, which assumes the user already knows enough to spot a mistake. Neither has any special knowledge of disks, so nothing stops a suggestion from targeting the wrong device.

Shell-GPT and Open Interpreter are open source and can use a local model, which suits lab machines. Open Interpreter can run whole tasks on its own, but it does this by letting the model write and execute arbitrary code. That is very flexible but is the opposite of what we want for operations that run as root. The graphical disk tools are safe and clear but are not available in a terminal or over SSH, cannot be used by other agents, and do not teach the user the underlying commands.

From this comparison we found a gap: none of these tools combines plain English input, a fixed set of checked disk operations, offline use, support for both Linux and Windows, and a display of the exact commands that ran. Our agent is designed to fill that gap for the narrow job of mounting and partitions.

References

[1] Lin, X. V., Wang, C., Zettlemoyer, L. and Ernst, M. D. (2018). NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System. Proceedings of LREC 2018.

[2] Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

[3] Agarwal, M. et al. (2021). NLC2CMD Competition: Translating Natural Language to Bash Commands. Proceedings of the NeurIPS 2020 Competition and Demonstration Track, PMLR 133.

[4] Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems 36.

[5] Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR).

[6] Ji, Z. et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 55(12).

[7] Anthropic (2024). Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

[8] Model Context Protocol Specification. https://modelcontextprotocol.io

[9] Linux man pages: mount(2), mount(8), losetup(8), lsblk(8), blkid(8). https://man7.org/linux/man-pages/

[10] Microsoft. Storage module cmdlets (Mount-DiskImage, Get-Disk, Get-Partition). https://learn.microsoft.com/powershell/module/storage/